

Báo cáo bài thực hành Caro AI

1. Mô tả bài toán

Chương trình xây dựng game cờ Caro giữa người chơi và máy tính trên bàn cờ 15x15. Máy tính chọn nước đi bằng thuật toán tìm kiếm đối kháng Minimax và phiên bản cải tiến Alpha-Beta pruning.

2. Luật chơi và điều kiện thắng

Hai bên đánh luân phiên vào các ô trống. Quân đen đi trước. Một bên thắng khi có 4 quân liên tiếp theo hàng ngang, hàng dọc hoặc một trong hai đường chéo. Không xét luật chặn hai đầu. Nếu bàn cờ đầy mà không có người thắng thì hòa.

3. Biểu diễn trạng thái bàn cờ

Bàn cờ được biểu diễn bằng mảng hai chiều boardMap[15][15]:

? `0`: ô trống.

? `1`: quân đen, cũng là người chơi MAX trong hàm đánh giá.

? `-1`: quân trắng, cũng là người chơi MIN trong hàm đánh giá.

Các thông tin phụ gồm currentI, currentJ, lastPlayed, emptyCells, boardValue, nextBound, nodes_visited và rollingHash.

4. Sinh nước đi hợp lệ

Hàm isValid(i, j) kiểm tra tọa độ nằm trong bàn cờ và ô chưa có quân. Để giảm không gian tìm kiếm, chương trình không duyệt toàn bộ bàn sau khi đã có quân, mà chỉ sinh các ô trống nằm gần quân đã đánh trong nextBound. Danh sách nước đi được sắp xếp theo điểm heuristic và độ gần trung tâm.

5. Kiểm tra trạng thái kết thúc

Sau mỗi nước đi, chương trình kiểm tra 4 hướng: ngang, dọc, chéo chính và chéo phụ. Nếu đếm được ít nhất 4 quân liên tiếp cùng màu thì trả về người thắng. Nếu không còn ô trống thì trả về hòa.

6. Thuật toán Minimax

Minimax được cài đặt trong source/AI.py. Nếu trạng thái là thắng, thua hoặc hòa thì trả về điểm kết thúc. Nếu đạt giới hạn độ sâu thì trả về điểm heuristic. Ở lượt MAX, thuật toán chọn giá trị lớn nhất; ở lượt MIN, thuật toán chọn giá trị nhỏ nhất. Thuật toán trả về nước đi tốt nhất, giá trị đánh giá, số trạng thái đã xét và thời gian chạy.

7. Thuật toán Alpha-Beta

Alpha-Beta dùng cùng hàm đánh giá và cùng độ sâu với Minimax. Thuật toán duy trì alpha là giá trị tốt nhất hiện tại của MAX và beta là giá trị tốt nhất hiện tại của MIN. Khi $\beta \leq \alpha$, nhánh còn lại bị cắt. Chương trình có thêm bảng chuyển vị đơn giản dùng Zobrist hashing để tái sử dụng giá trị đã tìm trong cùng lượt AI.

8. Hàm đánh giá trạng thái

Hàm đánh giá tính điểm từ góc nhìn quân đen (1). Các mẫu quan trọng:

? 4 quân liên tiếp: điểm rất lớn.

? 3 quân mở hoặc bị chặn một đầu: điểm cao.

? 3 quân có lỗ hổng: điểm trung bình cao.

? 2 quân mở: điểm nhỏ.

Điểm của quân trắng là điểm âm tương ứng, giúp MIN chọn nước làm giảm lợi thế của MAX.

9. Thiết kế trạng thái thử nghiệm

File performance_eval.py định nghĩa 5 trạng thái:

? Đầu ván.

? Giữa ván.

? AI có thể thắng ngay.

? Người chơi sắp thắng, AI cần chặn.

? Hai bên cùng có cơ hội tấn công.

Mỗi trạng thái được chạy với depth 1, 2 và 3.

10. Bảng kết quả thực nghiệm

Kết quả đầy đủ nằm trong ket_qua_thuc_nghiem.csv. Bảng tóm tắt tại depth 3:

Trạng thái	Minimax move	Minimax nodes	Minimax time	Alpha-Beta move	Alpha-Beta nodes	Alpha-Beta time	Giảm node
Đầu ván	(7,7)	98	0.039452s	(7,7)	28	0.011692s	71.43%
Giữa ván	(6,6)	14263	5.799513s	(6,6)	654	0.260702s	95.41%
AI có thể thắng ngay	(7,9)	10675	4.347338s	(7,9)	459	0.190813s	95.70%
Người sắp thắng cần chặn	(7,9)	10718	4.296740s	(7,9)	113	0.044616s	98.95%
Hai bên cùng tấn công	(10,7)	7707	3.074819s	(10,7)	415	0.170060s	94.62%

11. Nhận xét về số trạng thái và thời gian

Ở depth 1, Alpha-Beta gần như không giảm node vì cây tìm kiếm quá nông. Từ depth 2 trở đi, Alpha-Beta giảm rõ rệt số trạng thái đã xét. Ở depth 3, mức giảm node trong các trạng thái giữa ván thường trên 94%, kéo thời gian từ vài giây xuống dưới một giây.

12. Ảnh hưởng của độ sâu tìm kiếm

Depth càng lớn thì AI nhìn trước được nhiều phản ứng của đối thủ hơn. Ví dụ trạng thái cần chặn ở depth 1 chỉ dựa mạnh vào heuristic, còn depth 2 và 3 đánh giá thêm nước phản công của đối thủ. Đổi lại, số node của Minimax tăng rất nhanh; Alpha-Beta giúp depth 3 vẫn dùng được trong GUI.

13. Ưu điểm và hạn chế

Ưu điểm:

? Chạy được đủ bốn chế độ: Human vs AI, AI vs Human, AI vs AI, Human vs Human.

? Có cả Minimax và Alpha-Beta để so sánh công bằng trên cùng hàm đánh giá.

? Có ghi nhận nước đi, giá trị, depth, số node và thời gian.

? Sinh nước đi gần quân đã đánh nên tốc độ tốt hơn duyệt toàn bàn.

Hạn chế:

? Hàm đánh giá vẫn dựa vào pattern thủ công nên có thể bỏ sót thế cờ phức tạp.

? Chưa có iterative deepening hoặc giới hạn thời gian cứng cho từng nước.

? Chưa phân biệt nhiều cấp độ khó ngoài tham số depth.

14. Link repository GitHub của nhóm

Chưa có link repository GitHub của nhóm trong workspace. Khi nộp bài, cần bổ sung link repo theo mẫu tên mssv1_mssv2_mssv3_CaroAI.

15. Tài liệu tham khảo

? Đề bài `De\_so\_1.pdf`.

? Repo mẫu `<https://github.com/husus/gomokuAI-py>`: tham khảo cách tổ chức project, GUI Pygame và ý tưởng gom nhóm nước đi gần quân đã đánh.

? Repo mẫu `[https://github.com/MonHauVD/Caro\\_AI](https://github.com/MonHauVD/Caro_AI)`: tham khảo ý tưởng tổ chức chế độ chơi, benchmark và phân tích thực nghiệm.

Phản đã điều chỉnh so với repo mẫu: luật thắng 4 quân theo đề, hỗ trợ bốn chế độ chơi, sửa Minimax/Alpha-Beta để hoạt động cho cả quân đen và quân trắng, thêm benchmark 5 trạng thái và báo cáo thực nghiệm.